



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

12 · Pruebas y calidad

Documentación de sistema · Empresa Operativa Chile

Documento 12 de la serie

Versión del manifiesto 1.4.0 · commit 2b0e006

Análisis del 2026-08-27

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

12 · Pruebas y calidad

La conclusión, primero

153 pruebas en 13 suites, todas verdes, ejecutadas en esta revisión. Corren con el runner nativo de Node (`node --test`), sin framework, sin *mocks*, sin configuración y en **0,64 segundos**.

Y una segunda capa que importa más de lo que parece: **el repositorio verifica cosas que un test unitario no puede ver**. Que el APK lleve la app dentro. Que el build sea reproducible. Que la documentación no se haya desviado del código. Que ninguna acción de CI esté sin fijar. Esa capa es lo distintivo de este proyecto, y está en `ci.yml` y en los scripts de verificación, no en `tests/`.

El hueco real: la interfaz —las 14 vistas, 4.000 líneas de `apps/web/src/views/`— **no tiene pruebas de comportamiento**. Se verifica su presencia y su contrato, no lo que hace al pulsar un botón.

Comandos ejecutados y su salida real

Todo lo que sigue se ejecutó el **27-08-2026** sobre el commit `2b0e006`, en Windows 11 con Node 22 y pnpm 11.2.2. Ningún número de esta serie de documentos es una estimación.

Suite completa

```
node --test tests/*.test.mjs

i tests 153
i suites 0
i pass 153
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 642.8929
```

Los siete gates de sincronía y validación

Comando	Salida real	r c
<code>node scripts/build-rules.mjs --check</code>	Reglas embebidas sincronizadas (2026).	0
<code>node scripts/build-glossary.mjs --check</code>	Glosario sincronizado (54 términos).	0
<code>node scripts/build-guide.mjs --check</code>	Guía sincronizada (14 etapas, 2 diagramas).	0
<code>node scripts/build-shortcuts.mjs --check</code>	Atajos sincronizados (12).	0
<code>node scripts/build-presentation.mjs --check</code>	Presentación válida: 8 diapositivas, 28 minutos.	0
<code>node scripts/check-presentation.mjs</code>	Presentación: 8 diapositivas (713 KB), ≈28 min, pauta de 5 páginas (726 KB) – coincide con lo anunciado.	0
<code>node scripts/validate-rules.mjs</code>	Reglas 2026 validadas; última verificación 2026-08-09	0

Build y reproducibilidad

```
node scripts/build-all.mjs
```

```
apps/web/dist listo - 49 archivos, 7256 KB, build 630ed69fe514
Build completo. La aplicación está en apps/web/dist.
```

Segundo build seguido, comparando `build-info.json` :

```
BUILD REPRODUCIBLE: 630ed69fe514
```

```
{ "buildId": "630ed69fe514", "files": 49, "bytes": 7429748, "builtFrom": "apps/web/src + packages/" }
```

La CLI

Comando	Comprobación	Resultado
<code>venta --neto 100000</code>	<code>"iva": 19000</code>	✓
<code>rut 76.000.000-0</code>	<code>"valid": true</code>	✓
<code>f29 --ventas-netas 1000000 --compras-netas 300000</code>	<code>"vatPayable": 133000</code>	✓
<code>vencimientos --periodo 2026-08</code>	Cae en <code>2026-09</code>	✓ <code>2026-09-14</code> , <code>2026-09-21</code> , <code>2026-09-28</code> , dos de ellos con <code>shiftedFromWeekend: true</code>

El servidor local y sus defensas

Arrancado, verificado y detenido en esta revisión:

Comprobación	Resultado
<code>GET /</code> sirve la app	✓ contiene «Empresa Operativa Chile»
<code>GET /core/accounting-engine/index.mjs</code>	✓ contiene <code>f29Basic</code>
<code>GET ../../package.json</code>	404
<code>GET /%2e%2e%2f%2e%2e%2fpackage.json</code>	404
<code>GET /..%2f..%2fpackage.json</code>	404
<code>POST /</code>	405

Los tres vectores de escape son los mismos que ejecuta `ci.yml` ; el `405` lo añade esta revisión.

Las 13 suites

Suite	Pruebas	Qué cubre
<code>municipal-patent.test.mjs</code>	23	Base según etapa del negocio, deducciones, prorrateo, tope y piso en UTM, resolución de tasa y UTM
<code>capital.test.mjs</code>	19	Las seis magnitudes del capital, migración del campo heredado, <code>null</code> frente a <code>0</code> , validación del perfil
<code>tax-equity.test.mjs</code>	17	Elección de método, art. 41 frente a 14 D 3 (j), piso en cero, elegibilidad por régimen
<code>onboarding.test.mjs</code>	17	Las 14 etapas de la ruta, fases, pasos con trámite
<code>ayuda.test.mjs</code>	16	Términos, tooltips y coherencia de la ayuda contextual
<code>workspace.test.mjs</code>	14	Inmutabilidad del período, evidencia obligatoria, aislamiento real/sandbox, bitácora, export/import
<code>glossary.test.mjs</code>	11	Los 54 términos, categorías, búsqueda, «no confundir con»
<code>f29.test.mjs</code>	8	Arrastre del remanente, los dos modos de <code>f29Basic</code> , los tres vencimientos
<code>webapp.test.mjs</code>	8	El índice referencia lo que el build produce; el manifiesto apunta a iconos que existen; mismo <code>appId</code> en Android y Windows
<code>accounting.test.mjs</code>	6	Venta, compra, honorario, PPM, IDPC, asiento
<code>rut.test.mjs</code>	6	Dígito verificador, formatos, casos límite
<code>company-operations.test.mjs</code>	4	Tipos de operación, catálogo de constitución
<code>rules.test.mjs</code>	4	Un año sin reglas lanza ; años disponibles; procedencia
Total	153	

Las pruebas que valen por diez

Estas fijan reglas de negocio, no funciones. Si alguna se pone roja, algo se rompió en el producto y no en el código:

Prueba	Qué protege
<code>LoadRules(1999)</code> lanza	Que el sistema nunca calcule con la tasa del año equivocado
«un período cerrado es inmutable en las dos direcciones»	Que «cerrar el mes» signifique algo
«reabrir un período exige motivo y queda en la bitácora»	Trazabilidad por encima de inmutabilidad absoluta
«el remanente de crédito fiscal viaja entre períodos»	Que no se le cobre al usuario un IVA que no debe
«un paso de constitución no puede darse por hecho sin evidencia»	Que la app no mienta sobre trámites ante terceros
«una obligación no puede marcarse cumplida sin comprobante»	Idem
«el almacén web aísla real de sandbox en el mismo origen»	El invariante central del producto
«sembrar el sandbox sobre una empresa real está prohibido»	Que practicar no contamine lo real
«exportar e importar reproduce el espacio de trabajo completo»	Que los datos sean del usuario
«importar un archivo ajeno se rechaza»	
«toda mutación queda en la bitácora append-only»	Que la bitácora sirva como evidencia
«el diagnóstico marca error cuando hay obligaciones vencidas»	Que el semáforo no tranquilice de más

Cómo están escritas

```
node --test tests/*.test.mjs
```

Sin framework, sin `describe`, sin *mocks* y sin fixtures en disco: `createMemoryStore()` da un espacio de trabajo limpio en cada prueba. La consecuencia práctica es que **no hay nada que instalar para ejecutarlas** — sólo Node ≥ 20 — y que no existe una capa de dobles que pueda diverger del comportamiento real.

`node --test` reporta `suítes 0` porque las pruebas son llamadas planas a `test()`, sin agrupar.

La otra capa: verificar lo que un test unitario no ve

Es lo que distingue a este repositorio, y merece su propia tabla.

Verificación	Script / paso	Qué atrapa que un test no atraparía
El APK lleva la app dentro	<code>verify-apk.mjs</code> , 12 comprobaciones abriendo el ZIP	Un APK vacío compila perfectamente : la WebView arranca, muestra una pantalla en blanco y todo sigue en verde
La interfaz está completa antes de sellarla	<code>desktop.yml</code>	Tauri comprime los recursos dentro del binario: después ya no se pueden contar
El ejecutable arranca y sigue vivo	<code>desktop.yml</code> , 12 segundos	Fallos de arranque por DLL o recursos ausentes
El binario no es sospechosamente pequeño	<code>desktop.yml</code> , umbral de 1 MB	Un empaquetado incompleto
El build es reproducible	<code>ci.yml</code> , dos builds y <code>diff</code>	Que el artefacto publicado no sea el que se probó
El servidor no escapa de <code>dist/</code>	<code>ci.yml</code> , 3 vectores	Una regresión en <code>resolveSafe</code>
La documentación no se desvía del código	5 gates <code>--check</code>	Que el glosario, la guía o los atajos digan una cosa y la app haga otra
Las acciones están fijadas a SHA	<code>ci.yml</code>	Una etiqueta móvil que cambie de contenido
Los workflows son válidos	<code>actionlint</code>	Errores de sintaxis que sólo se verían al fallar
Cero dependencias de producción	<code>security.yml</code>	Que el argumento del proyecto deje de ser cierto por descuido
Sin datos reales en el repositorio	<code>security.yml</code>	El riesgo nº 1 del modelo de amenazas
La presentación tiene lo que anuncia	<code>check-presentation.mjs</code> , abre los PDF y cuenta páginas	Un PDF vacío o desactualizado

Integración continua

Seis workflows. Los tres primeros son los que corren solos:

Workflow	Disparadores	Trabajos	Duración máx.
ci.yml	push y PR a main , manual	pruebas (matriz), servidor , workflows	12 / 8 / 5 min
security.yml	push, PR, lunes 06:00 UTC, manual	codeql , datos	20 / 5 min
pages.yml	push a main , manual	publicar	10 min
android.yml	manual, workflow_call	apk	30 min
desktop.yml	manual, workflow_call	windows	45 min
release.yml	etiqueta v* , manual	android + windows → publicar	10 min

La matriz de ci.yml : Ubuntu × Node 20 y 22, más Windows × Node 22 (Windows con Node 20 está excluido). Windows entra en la matriz porque es la plataforma de escritorio del producto: si las rutas se rompen ahí, se rompe el instalador.

ci.yml no tiene filtros paths , y está comentado por qué: con filtros, un commit que sólo toca documentación no ejecuta nada y la rama queda sin estado, que no es lo mismo que estar en verde.

android.yml y desktop.yml no corren en cada push —tardan 30 y 45 minutos— sino a mano o llamados por release.yml . INFERENCIA : la consecuencia es que una regresión que sólo rompa el empaquetado no se detecta hasta que alguien publica.

Análisis estático, linting y formato

Herramienta	Alcance	Estado
CodeQL	JavaScript/TypeScript, conjunto <code>security-and-quality</code>	✓ Activo en push, PR y semanalmente
actionlint	<code>.github/workflows/</code>	✓ Activo en cada push
Verificación de pinning	<code>.github/workflows/</code>	✓ <code>grep</code> que falla si falta un SHA
ESLint	—	✗ NO IDENTIFICADO : no hay configuración
Prettier	—	✗ NO IDENTIFICADO
TypeScript / <code>checkJs</code>	—	✗ No hay <code>tsconfig.json</code>
Clippy / rustfmt	Rust	✗ NO IDENTIFICADO en los workflows
Cobertura	—	✗ No se mide
Dependabot	—	✗ NO IDENTIFICADO

La ausencia de ESLint y Prettier no ha producido un código inconsistente —el estilo del repositorio es notablemente uniforme—, pero eso hoy lo sostiene una sola persona, no una herramienta.

El tipado sí existe, en JSDoc: `@param {{ store: object, mode?: 'real'|'sandbox' }}` y similares están por todo el código. Nadie los comprueba, porque no hay `checkJs`.

Cobertura observable

No se mide, así que lo que sigue es un mapa cualitativo hecho leyendo qué toca cada suite.

Área	Archivos	Cobertura aparente	Evidencia
chile-tax-rules	index.mjs	Alta	4 pruebas, incluida la que más importa
accounting-engine	index.mjs , tax-equity.mjs , municipal-patent.mjs	Muy alta	6 + 17 + 23 + 8 pruebas
company-operations	workspace.mjs , capital.mjs , rut.mjs , store.mjs	Alta	14 + 19 + 6 + 4
glossary , onboarding , shortcuts		Alta	11 + 17 + 16, más 4 gates <code>--check</code>
Contrato de la web	index.html , manifiesto, iconos	Media	8 pruebas, de presencia y contrato, no de comportamiento
Vistas	apps/web/src/views/*.js (14)	✗ Ninguna	Ninguna prueba las ejecuta
lib/dom.js , state.js , platform.js		✗ Ninguna directa	Ni siquiera el escapado, que es un control de seguridad
server.mjs		Media	Sin prueba unitaria; sí verificación en CI
node-store.mjs		Baja	Se ejercita indirectamente por la CLI en CI
Backend Rust	lib.rs	Baja	2 pruebas para 4 comandos; no ejecutadas en esta máquina
Scripts de build	scripts/*.mjs (13)	✗ Ninguna	Se ejercitan al correr; no hay pruebas propias

Módulos sin pruebas, y qué costaría cubrirlos

Módulo	Riesgo de no cubrirlo	Dificultad
<code>lib/dom.js</code> · <code>esc</code> y <code>html</code>	Alto: es el control anti-XSS del sistema	Trivial: son funciones puras. Media docena de aserciones
<code>lib/platform.js</code> · <code>hydrateFromDisk</code>	Alto: decide si se pisan datos vivos con un espejo viejo	Media: hay que simular <code>__TAURI__</code> y <code>localStorage</code>
<code>store.mjs</code> · <code>createWebStore</code> con cuota llena	Alto: hoy la escritura se pierde en silencio	Media: simular que <code>setItem</code> lanza
Vistas	Medio: un fallo se ve de inmediato al usar la app	Alta: exige un DOM
<code>resolveSafe</code> como unidad	Bajo: ya está probado en CI de punta a punta	Trivial
Saneos de <code>filename</code> en Rust	Medio	Trivial: al lado de las dos que ya existen
Scripts de build	Bajo: fallan ruidosamente	Media

Propuesta priorizada de pruebas faltantes

Ordenada por relación entre lo que protege y lo que cuesta. **Es una recomendación; no se implementó nada.**

#	Prueba	Por qué primero
1	<code>esc()</code> y <code>html</code> escapan <code><</code> , <code>></code> , <code>&</code> , <code>"</code> , <code>'</code> ; <code>raw()</code> no	Control de seguridad, coste trivial
2	<code>createWebStore</code> cuando <code>setItem</code> lanza <code>QuotaExceededError</code>	Hoy se pierde una escritura en silencio
3	Saneos de <code>filename</code> en <code>export_file</code> (Rust)	Al lado de las dos pruebas que ya existen
4	<code>save_workspace</code> rechaza un payload que no es JSON	Idem
5	<code>resolveSafe</code> como prueba unitaria, con los tres vectores	Deja de depender de que el servidor arranque
6	<code>hydrateFromDisk</code> no pisa un dato vivo con uno del espejo	La carrera está comentada en el código, no probada
7	Coherencia entre las dos <code>version</code> (<code>package.json</code> y <code>tauri.conf.json</code>)	Un gate de una línea
8	Al menos una vista: renderizar <code>panel</code> sobre un almacén de memoria	Rompe el cero de cobertura de la interfaz
9	<code>capitalHistory()</code> con dos ejercicios cerrados	La cadena de tres años es la lección del producto
10	Un segundo año de reglas (<code>rules/2027.json</code> de prueba)	El mecanismo nunca se ha ejercitado con dos archivos

Criterios de aceptación

Reconstruidos a partir de lo que CI exige; el repositorio no los enuncia en un documento aparte.

Para que un cambio entre a `main`:

1. Las 153 pruebas pasan en Ubuntu (Node 20 y 22) y Windows (Node 22).
2. Los cinco gates de sincronía documental pasan.
3. `validate-rules.mjs` pasa.
4. El build completo funciona **y es reproducible**.
5. La CLI responde con los valores esperados.
6. El servidor sirve la app y bloquea los tres vectores de escape.
7. `actionlint` pasa y todas las acciones están fijadas a SHA.
8. CodeQL no encuentra nada nuevo.
9. No hay datos reales, certificados ni claves en el repositorio.
10. `dependencies` sigue vacío.

`CONTRIBUTING.md` añade uno que no es automatizable y es el más importante: **una tasa que se toca exige su fuente oficial y su fecha de verificación**.

Lo que no pudo verificarse

Aspecto	Por qué
<code>cargo test --release</code>	Sin toolchain de Rust en la máquina de análisis
Compilación del APK y <code>verify-apk.mjs</code> sobre un APK real	Requiere JDK 21, SDK de Android y Gradle
Compilación de los instaladores de Windows	Requiere Rust y <code>@tauri-apps/cli</code>
Resultados de CodeQL	Viven en la pestaña Security del repositorio
Comportamiento de las vistas	No hay pruebas y no se ejecutó la interfaz en esta revisión
Que CI esté hoy en verde en GitHub	No se consultó el estado remoto